CAS PCBA 8: ADAPTIVE COGNITIVE CIRCUIT ENGINEERING WITH TYPED, FAIL-CLOSED DESIGN RULES AND A PRESERVED ADVERSE SOURCE AUDIT

LITTLE HENDERSON

Abstract. CAS PCBA 8 is a typed, provenance-bearing research architecture for describing and checking circuit-shaped cognitive-system models. "Circuit" names an engineering organization—typed components, ports, nets, state annotations, calibration references, deterministic traces—and does not assert a physical electrical implementation or identify model quantities with voltage, current, or resistance. The research question is whether a circuit-shaped modeling language can make cognitive-system hypotheses more explicit, type-safe, replayable, and falsifiable *without* converting an analogy into a physical, causal, safety, or product claim.

This paper develops the affirmative software-contract answer in full. We specify the type system (state literals, ND-cells, strictly positive simplex addresses, tagged exact values, one immutable decimal context); the deterministic design-rule checker with its closed 35-code `PCBA8-*` diagnostic catalog across model, semantics, parameters, states, bias, nets, test points, calibration, feedback, safety, and TKS-adapter classes; the narrow deterministic simulator and its canonical trace bindings; the Fisher–Rao and projector geometry profiles with their disclosed numerical limits; and the certified-transition adapter into the TKS v8 finite kernel. We then present the adverse source audit at full evidential resolution: the consolidated CCDL grammar admits only seven `circuit_item` alternatives and a two-alternative board body, while the five published reference boards use dozens of reserved statement leaders absent from the grammar—four boards are direct syntactic counterexamples, itemized here with page anchors and observed forms—and the published Cognitive Safety Board clamps its red stage at 0.1 upstream of a 0.9 fuse threshold, making the protective crossing unreachable under the declared clamp semantics; we formalize that unreachability argument. Bounded software evidence (82 CAS PCBA v8 test methods within an integrated 208-method, 272-subtest, 54-artifact Gate 4 receipt) establishes tested software behavior only. Threats to validity, the claim quarantine, and a dependency-ordered research program complete the paper.

Contents

1. Introduction 2
1.1. The research question 2
1.2. Three load-bearing separations 2
2. Versioned source basis 3

Date: August 7, 2026.

Research paper v0.2, 2026-08-07 (supersedes white paper v0.1). Artifact status: DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. Engineering baseline: `CAS_PCBA_Engineer.pdf`, 743 pp., source ID SRC-PCBA-ENGINEER-20260726, SHA-256 `294320B39FA521EAC086E99E2D9DB913E2B6CFF0754FF82DB1EF966424566274`. Controlling artifacts: `CAS_PCBA_V8_SOURCE_AUDIT_AND_SPEC_v0.1.md` (SHA-256 `19707B92...`); `PCBA_CCDL_CONSTRUCT_COVERAGE_v0.1.json` (SHA-256 `5BB797EC...`); Gate 4 review matrix and artifact manifest v0.1; publication delta Paper 6 at base commit `a1cd9901`. Implementation `src/cas_pcba_v8/` (14 modules). Drafting this paper promotes no claim; independent systems, safety, regulatory, clinical, and formal review remains open.

3. Typed semantic and board model 3
3.1. Semantic identity 3
3.2. State types 3
3.3. Values and arithmetic 3
3.4. Board structure 4
3.5. Geometry profiles 4
4. The design-rule checker 4
5. Narrow deterministic simulation 5
6. The adverse source audit 5
6.1. CCDL grammar coverage 5
6.2. The CSB known-fault board 6
6.3. Claim quarantine 7
7. TKS v8 and Noetic Topos 2 boundaries 7
8. Bounded software evidence 7
9. Threats to validity 8
10. Dependency-ordered research program 8
11. Conclusion 9
References 9

1. Introduction

1.1. **The research question.** The engineering question is not whether cognition *is* electronics. It is:

> Can a circuit-shaped modeling language make cognitive-system hypotheses more explicit, type-safe, replayable, and falsifiable without converting an analogy into a physical, causal, safety, or product claim?

The current candidate supports a qualified affirmative at the software-contract level. Its contributions are: (1) a typed board model separating semantic state, geometry, signal kind, unit, component contract, calibration metadata, and evidence provenance; (2) a deterministic design-rule checker (DRC) with fail-closed diagnostics; (3) a narrow trace-emitting reference simulator for declared deterministic component profiles; (4) a source- and profile-bound adapter from certified finite component tables to the TKS v8 finite reference kernel; (5) a source audit that preserves, rather than silently repairs, the incomplete CCDL grammar and the CSB threshold contradiction; and (6) an evidence discipline distinguishing green software tests from experiments, measurement validation, safety evidence, and authority.

1.2. **Three load-bearing separations.**

(1) **Model versus world.** A simulated transfer rule is behavior of the declared software profile, not evidence that a cognitive or physical system obeys it.
(2) **Design value versus measurement.** A locally supplied parameter or calibration label remains structural metadata until authenticated measurement evidence and uncertainty are supplied.
(3) **Analogy versus identity.** Names borrowed from circuits do not make CAS quantities SI electrical quantities; physical voltage, current, resistance, charge, flux, and power are outside the candidate unless a separately typed, instrumented physical profile is proposed and validated.

The engineering literature supports this discipline at the level of *method*, not CAS validity: VHDL demonstrates the value of machine- and human-readable formal hardware description [1]; Statecharts demonstrate the need for explicit state, hierarchy, concurrency, and communication semantics [2]; and simulation verification is a distinct activity from operational validation [6]. None of these sources validates the CAS ontology or its cognitive interpretation.

2. Versioned source basis

The immediate engineering baseline is the 743-page owner-supplied CAS PCBA Engineer build, source ID SRC-PCBA-ENGINEER-20260726. Its internal changelog label predates the supplied build, so the candidate uses the source ID plus exact hash and does not invent a public semantic version. The book is an engineering reference; its own disclaimer, the Phase-0 authority matrix, and this candidate all deny safety certification and physical telemetry.

Downstream read-only context: the TKS v8 core formal specification (finite-arrow boundary) and the Noetic Topos 2.0 specification (ambient/resolved evidence boundary). The Gate 5 preregistration specification supplies no experiment to this paper; at the bound commit it is OPEN / NOT STARTED / RING 4 / NOT LICENSED / HELD / NOT FROZEN, and nothing here promotes it.

3. Typed semantic and board model

3.1. **Semantic identity.**

CAS Core 8 supplies the structural reference tuple (namespace, local_id, version, source with TKS Foundations, TKS Worlds, CAS Drives, Domains, Registers, States, and Noetic Topos objects in disjoint namespaces and no silent identity across them. This prevents a board adapter from manufacturing a semantic theorem because two objects share a label or position.

3.2. **State types.**

Definition 3.1 (State literals and cells)**.**

$$\text{StateLiteral} = \text{Noetic} \times \text{Domain} \times \text{Drive}_{\text{opt}} \times \text{Register}_{\text{opt}}, \qquad \text{NDCell} = \text{Noetic} \times \text{Domain}.$$

The projection StateLiteral $\to$ NDCell forgets Drive and Register; it is explicitly many-to-one and is not an isomorphism.

Definition 3.2 (Simplex address)**.** A $\text{SimplexAddress}_{40}$ is a strictly positive normalized profile over the forty canonical ND-cells: $p \in \mathbb{R}^{40}$ with $p_k > 0$ for all k and $\sum_k p_k = 1$. It is neither a one-hot cell nor an element of the finite TKS kernel; positivity is validated strictly.

3.3. **Values and arithmetic.**

Rule 3.3 (Tagged exact values)**.** Finite carrier values use tagged exact type-plus-value identity: integer 1, Boolean *true*, and string `"true"` are distinct throughout DRC, execution, traces, and adapters.

Rule 3.4 (One decimal context)**.** All decimal operations run under one immutable, versioned context created from primitive constants for each operation; ambient caller precision is irrelevant, and the context identity enters reports and results. IEEE 754 [4] is an external reference for explicit arithmetic control; the candidate's decimal profile remains its own software contract.

3.4. **Board structure.** A board contains immutable, identity-bearing records for: components and declared kinds; typed ports (direction, signal kind, unit); typed parameters and finite-state annotations; nets and endpoints; test points and calibration references; declared feedback loops; source and research-boundary metadata; and deterministic identity material. Board identity includes the complete canonical structure, so any structural edit changes the board's digest. Public operations derive and recheck the fixed research boundary rather than trusting a shared mutable object.

3.5. **Geometry profiles.** The mixed-simplex baseline retains a named Fisher–Rao profile; information geometry supplies the mathematical setting for probability-simplex metrics [8] but does not identify a learned model state with a CAS state. The disclosed binary64 implementation is a numerical pseudometric at its resolution—not a proof that all distinct profiles are separated. Projector geometry is a separate, optional research profile: it caps and versions its tolerance, records validation residuals, and binds the canonical matrix, basis/source, dimension, rank, and tolerance in one digest. The registered projector Gram-kernel result requires common rank $d > 0$ and does not establish latent cognitive capacity; the stronger CAS capacity transfer retains a proof gap and is governed solely by the formal-mathematics Paper 3 record at its own commit.

4. The design-rule checker

The DRC computes one deterministic diagnostic report under the closed `PCBA8-*` catalog. Errors fail the board; warnings never become an implicit clean result. The complete catalog:

Class	Codes	Diagnostics
Model	`MDL-001/002`	invalid board metadata; duplicate reference
Semantics	`SEM-001/002`	unknown component kind; unresolved semantics
Parameters	`PAR-001/002/003`	unknown; missing; invalid parameter
States	`STATE-001/002/003`	unknown; missing; invalid state binding
Bias	`BIAS-001`	bias-domain incoherence
Nets	`NET-001–009`	unknown endpoint owner; unknown port; invalid driver count; missing net load; sig
Test points	`TP-001/002/003`	unknown test-point net; missing safety test point; safety test point not marked
Calibration	`CAL-001–004`	unknown; invalid; unit-mismatched; missing test-point calibration
Feedback	`FBK-001–004`	undeclared feedback; feedback without delay; unsafe feedback gain; invalid feedbac
Safety	`SAF-001/002/003`	unreachable safety threshold; unsafe fuse action; unresolved safety analysis
TKS adapter	`TKS-001`	invalid finite transition

Check Class 4.1 (Structural integrity)**.** Unique nonblank identities for components, ports, nets, test points, calibrations, and feedback declarations; component kinds and exact parameter keys from a closed catalog; missing or unknown semantics fail closed (`MDL`, `SEM`, `PAR`, `STATE` classes).

Check Class 4.2 (Typed connectivity)**.** Every endpoint resolves to a declared owner and port; every net has exactly one typed driver; driver and load signal kinds and units agree under a closed admissible signal-kind/unit table; required inputs connect or have an executable default in the selected simulator profile; multiple drivers, missing loads, multiply-connected ports, and unsupported signal/unit pairs receive distinct `NET` diagnostics.

Check Class 4.3 (Calibration and probe integrity)**.** Executable test points require versioned calibration references with profile-digest shape, instrument and protocol identity fields, uncertainty, measurement kind, and net-unit compatibility. A locally constructed calibration

record cannot make itself Evidence-Packet eligible. Safety-marked paths require explicitly marked probes; a probe's presence does not establish safety (`TP`, `CAL` classes).

Check Class 4.4 (Feedback analysis)**.** The checker builds a directed component graph and computes strongly connected components (classical algorithms [3]). Each realizable declared cycle requires an exact feedback declaration, a declared unit delay, an existing dimensionless calibration, and a conservative sub-unity gain bound; explicit low-gain exceptions are declared, and stateful or stochastic loops are unsupported. Unsupported or resource-bounded analysis yields `SAF-003` (unresolved), never a pass (`FBK` class).

Check Class 4.5 (Threshold reachability)**.** For supported activation paths, conservative upper bounds propagate through causes, bias/gain stages, clipping, and cutoffs. A safety-critical fuse whose threshold exceeds the resolved upstream bound receives `SAF-001` (UNREACHABLE SAFETY THRESHOLD); an unresolved bound is not treated as reachable.

5. Narrow deterministic simulation

Simulation is admitted only for supported component profiles with declared ports, transfer rules, clipping, statefulness, and time semantics. The current profile topologically orders the non-delay graph, uses declared unit-delay behavior to break admitted feedback, and emits a canonical trace binding: board and component identities; the exact DRC report identity; component catalog and decimal-context identities; simulator profile and source baseline; typed input samples and outputs; component events and tick linkage; test-point observations; measurement IDs with parent/child derivations and calibration references; and the fixed Ring-4 research boundary.

Measurement and result construction validates digest form, unique IDs, child existence and derivation acyclicity, tick/trace linkage, units, and measurement-kind eligibility. Locally asserted calibration metadata is structural/test-only and cannot create authenticated Evidence-Packet eligibility. Unsupported behavior fails closed. The record discipline resembles the entity/activity/derivation separation of W3C PROV [5] without claiming PROV-O conformance.

Claim Boundary 5.1. The correct interpretation of a green fixture is: *the software performed the declared typed transition and produced the declared trace for the exact fixture under the exact profile.* It is never: the fixture proves a cognitive mechanism, a causal relation, a physical circuit law, a safe refusal mechanism, or a product capability. Simulation validates the implementation against declared fixtures; it does not establish that a cognitive or model system follows the simulated law.

6. The adverse source audit

The audit's method is bounded and disclosed: text extraction located candidate pages; every cited page was rendered from the source PDF and visually inspected; the comparison is between published surface syntax and the consolidated EBNF—it is not a parser implementation or a semantic-validation result.

6.1. CCDL grammar coverage.

Adverse Finding 6.1 (The formal grammar does not cover its own corpus)**.** *The language reference reserves many Part-IV statement leaders (PDF p. 578; book p. 562), including* `rail`, `dial`, `spec`, `constraint`, `sequence`, `route`, `derate`, `monitor`, `alert`, `hold`,

use, force_state, on, charge, discharge, refresh, scan, and select; prose tables describe select, scan, interrupt, and a verification sublanguage, and expressly say a form missing from the appendix should be treated as an erratum (PDF pp. 591–592). Yet the consolidated structural EBNF (PDF pp. 598–599) limits circuit_item to seven alternatives—io_decl, node_decl, cjt_decl, component_decl, feedback_decl, probe_decl, let_decl—and a board_block body to layer_decl or via_decl only. Against the five published reference boards, the machine-readable coverage record concludes FORMAL EBNF DOES NOT COVER PUBLISHED FIVE-BOARD CORPUS*, with four direct counterexamples and one bounded non-counterexample:*

Board	*Pages (PDF)*	*Observed forms outside the grammar*	*Verdict*
CCB-1-CSB	*606–608*	*board body contains* `input`, `output`, `node`, `cjt`, `diff`, `mux`, `filter`, `feedback`, `probe` — *not* `layer/via`	*counterexample*
CCB-2-COB	*613–614*	*leaders fit published circuit-item/component prefixes*	*no direct counterexample (bounded audit)*
CCB-3-CDAB	*618–619*	`mirror bias_mirror: replicate(shared_bias) -> [cjt_l, cjt_r]` *is not a* `type_name<arg_list>` *component expression*	*counterexample*
CCB-4-CMB	*623–624*	`charge`, `select`, `discharge`, `refresh`, `scan` *have no* `circuit_item` *alternative*	*counterexample*
CCB-5-CPMB	*628–632*	`rail`, `use`, `spec`, `dial`, `constraint`, `monitor`, `on`, `derate`, `route`, `alert`, `hold`, `sequence`, `gate`, `force_state` *have no alternative*	*counterexample*

The corpus-level conformance claim therefore fails, without overstating the result for the one bounded listing.

Rule 6.2 (Candidate grammar rule)**.** The v0.1 implementation declares only its programmatic Python object model and does not claim to parse the complete CCDL book language. Any later grammar must publish an exact construct-coverage matrix, reject unsupported forms, and round-trip every admitted board canonically.

6.2. **The CSB known-fault board.**

Adverse Finding 6.3 (The published safety fuse cannot trip)**.** *The Chapter 18 Cognitive Safety Board listing gives a red-stage saturation clamp* $\text{sat} = 0.1$ *and a downstream fuse threshold* 0.9 *(PDF p. 342; repeated in Appendix D at PDF p. 608). The same listing uses non-finite gain* $\beta = \infty$ *despite finite grammar and validator rules. The Appendix-D note (PDF p. 610) says the infinite gain is an idealization rejected by validators and directs the implementer to a large finite value—while retaining the* $\text{sat} = 0.1$ *clamp and without reconciling it with the* 0.9 *threshold. The book supplies no fail-closed latch state machine showing that an open fuse preserves rather than suppresses the refusal path.*

Proposition 6.4 (Unreachability under the declared semantics)**.** *Under the stated clamp semantics—saturation as an output plateau bounding the red stage above by* 0.1*—and with every downstream stage on the declared path monotone and non-amplifying under the published parameters, the fuse input never exceeds* 0.1*; since* $0.1 < 0.9$*, the threshold crossing*

is unreachable, and the DRC's conservative bound propagation (Check Class 4.5) necessarily flags `SAF-001`*.*

Proof. Let u denote the red-stage output; the clamp gives $u \leq 0.1$ pointwise. Each declared downstream stage s satisfies $s(x) \leq x$ for $x \geq 0$ under the published parameters (clipping and cutoff stages are non-increasing in bound; the published path contains no amplifying stage between clamp and fuse). By induction along the path, the fuse input is bounded by 0.1. A threshold event requires input ≥ 0.9. □

Remark 6.5 (Disposition). Saturation is a plateau, not a latch. The unmodified parameters are retained as a known-fault fixture that the DRC must flag—valuable negative evidence about the source and a permanent regression guard for the checker, not a validation of the board's safety function. Any repair is a separately versioned design requiring threshold reachability, finite parameters, latch state transitions, fuse truth tables, deterministic traces, hazard analysis, and independent safety review.

6.3. **Claim quarantine.** The following remain hypotheses or prohibited product inferences: exact thought/transistor isomorphism; shared four-region identity; overload-induced semantic involution; Fisher–Rao causal gain; physical electrical identity for CAS quantities; semantic correctness inferred from a response-region label; safety effectiveness, certification, regulatory compliance, or clinical use; complete compiler conformance; production readiness; and disclosure or license authority from a self-hash or generated receipt. Historical AI-authored, self-hashed licensing receipts are not an authenticated reviewer or legal authority.

7. TKS v8 and Noetic Topos 2 boundaries

Only a certified total, finite, deterministic, stateless, non-stochastic component transition may be adapted to a TKS v8 finite arrow. A typed sequence becomes a semigroupoid path; only an endomap has powers, so feedback powers require a certified endomap. The adapter returns an immutable certificate binding the exact TKS kernel bytes and profile, the PCBA board, component, and transition identities, the tagged carrier, the semantic profile, and the fixed boundary; ambient module substitution is rejected, and violations raise `TKS-001`. The adapter proves at most type-correct representation in the finite reference kernel; it carries no automatic CAS semantic theorem. Continuous simplex profiles, response curves, instrumented observations, stateful devices, stochastic kernels, and memristive history variables remain outside the finite arrow and require distinct types and comparison maps rather than coercion.

From Noetic Topos 2.0 the candidate inherits the ambient-versus-resolved evidence distinction: raw or partially resolved records may be treated as external evidence associated with the ambient construction, but a marked cover does not make a measurement true, and records do not enter resolved semantics until restriction and gluing under one calibration profile are defined and tested.

8. Bounded software evidence

At the bound commit, the integrated Gate 4 receipt records 54 hash-bound artifacts and three green focused suites:

Suite	Methods	Suite test-ID SHA-256
CAS Core v8	61	`E2AF5014DE28DD0702211EF32A2757B99B869A13FAFBB0B8D1B18A50A9F8FEF5`
CAS PCBA v8	82	`D5F776C5F15758CD390470B5A542CCA110784DC5EBB9807B6BE41B2C80453470`
CAS-Ohm v1	65	`2E81A685961286D944894A22EB93345B97211B4433B6C71EAF628D84CD8EB1EF`

The combined focused run comprises 208 test methods and 272 parameterized subtests; 11 claim records all Ring 4 and NOT LICENSED; 53 review criteria with 20 open; four direct EBNF counterexamples; eight CAS-Ohm source conflicts; and an integrated Gate 1 pass. The repository-wide collector was *not* clean at the bound receipt—four pre-existing package-path namespace collisions after 1,200 collected items—so the focused suites cannot support a repository-wide verification claim. These numbers are historical, source-bound software receipts, not experiments in this publication lane.

9. Threats to validity

Construct validity. No experiment establishes that the selected CAS types correspond to cognitive mechanisms or latent model structure; the circuit organization may aid design and audit without being an ontology of cognition.

Internal validity. The DRC and simulator have bounded regression suites, not proofs of all paths. Property-based/random generation [7], fuzz testing, independent implementations, and a separately sourced mutation-testing campaign remain open. The full CCDL parser does not exist.

External validity. There is no live calibrated model, physical board, instrument chain, held-out replication, or multi-model evaluation in the admitted evidence; cross-model and cross-context generalization are open.

Numerical validity. The fixed decimal profile controls candidate arithmetic; the disclosed binary64 Fisher–Rao implementation is a numerical pseudometric at its resolution. Independent numerical audit and tolerance analysis remain open.

Causal validity. Graph connectivity, gain composition, prediction, and residual localization do not establish causation; causal claims require explicit models, identification assumptions, and interventions [9]. None is admitted.

Safety and governance. Finding 6.3 blocks any safety-effectiveness inference. Hazard analysis, independent safety engineering, regulatory and clinical review, authenticated claim authority, canon admission, disclosure approval, and product acceptance are all open.

10. Dependency-ordered research program

(1) Specify a complete versioned CCDL grammar and a golden five-board corpus.
(2) Implement canonical parse/serialize round trips with fail-closed unsupported syntax.
(3) Define separate capacitor, PLL, memory, power, feedback, stochastic, and memristive state/time profiles.
(4) Run a property-based/random-generation campaign over DRC and simulation invariants [7]; separately specify and source any mutation-testing campaign.
(5) Independently reproduce the exact source and test bundle in a hermetic toolchain.
(6) Preregister calibration and held-out validation through the Gate 5 control kernel, without promoting Gate 5 status.
(7) Require executor/auditor numerical agreement, falsifiers, negative outcomes, and independent replication.

(8) Complete internal cross-review against stable hashes; then commission external formal, systems, experimental, safety, and claim-language review before any submission or use claim.

11. Conclusion

CAS PCBA 8's value is not that it proves cognition is electrical—it does not and cannot at this stage. Its value is architectural and evidential: informal pathway diagrams become typed, inspectable, versioned software objects; every executable claim is bound to a profile, source, calibration record, and trace; and negative evidence is preserved as first-class findings with page-anchored evidence rather than patched away. A grammar that does not cover its own corpus (Finding 6.1) and a safety board whose fuse cannot trip (Finding 6.3, Proposition 6.4) are recorded, formalized, and guarded by the checker. The framework makes whole categories of error observable earlier—scale and unit substitution, ambiguous crosswalks, incomplete grammar claims, multiple-driver and unresolved-port defects, unreachable threshold paths, undisciplined feedback, fabricated measurement lineage, and software receipts misrepresented as authority. That is a legitimate engineering contribution exactly as long as its boundary remains as explicit as its checks.

References

[1] IEEE Std 1076-2019, *VHDL Language Reference Manual*.
[2] D. Harel, "Statecharts: a visual formalism for complex systems," *Sci. Comput. Program.* 8 (1987), 231–274.
[3] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.* 1 (1972), 146–160.
[4] IEEE Std 754-2019, *Floating-Point Arithmetic*.
[5] W3C, *PROV-O: The PROV Ontology*, 2013.
[6] R. G. Sargent, "Verification and validation of simulation models," *J. Simulation* 7 (2013), 12–24.
[7] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," *ICFP* (2000).
[8] S.-i. Amari, *Information Geometry and Its Applications*, Springer, 2016.
[9] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed., Cambridge University Press, 2009.